

TinyKV Project 1 与 Project 4

代码变更及测试点分析

生成日期：2026-07-27

1. 分析范围与结论

本报告只分析 Project 1 (StandaloneKV) 和 Project 4 (Transactions) 的实现，不把当前工作区中的 Project 2、Project 3 修改计入这两部分。

- Project 1 的对比范围：提交 7a7d26b 到 8ae6dd7 (Project1_PASS)。
- Project 4 的对比范围：以 Project 1 提交 8ae6dd7 为基线，对比当前工作区中的四个事务相关文件。
- Project 1 共修改 2 个文件，增加 150 行、删除 21 行。
- Project 4 共修改 4 个文件，增加 626 行、删除 32 行。
- 本报告覆盖 71 个正式测试点：Project 1 为 10 个，Project 4A 为 15 个，Project 4B 为 24 个，Project 4C 为 22 个。

总体上，Project 1 完成了从 Raw API 到单机 Badger 存储的完整读写链路；Project 4 在此基础上完成了三列族 MVCC、两阶段提交、事务状态检查、批量回滚、锁解析和事务扫描。现有测试对课程要求覆盖较强，但通过测试不能严格证明实现不存在任何问题，尤其不能替代崩溃恢复、真实并发、分布式 Region 错误和故障注入测试。

2. Project 1: StandaloneKV

2.1 代码改动概览

文件	增加	删除	核心职责
kv/storage/standalone_storage/standalone_storage.go	49	10	Badger 生命周期、Reader、Write、列族访问
kv/server/raw_api.go	101	11	RawGet、RawPut、RawDelete、RawScan 及错误映射

2.2 standalone_storage.go 的改动与理由

1. 持有并初始化 Badger 数据库

StandAloneStorage 新增 db *badger.DB 字段，构造函数使用配置中的 DBPath 调用 engine_util.CreateDB 打开数据库。Start 保持为空操作，Stop 负责关闭数据库。

理由：

- StandaloneKV 必须把数据持久化到磁盘，而不是仅保存在进程内存中。
- 数据库在构造阶段即准备完毕，所有 Reader 和 Write 共享同一实例。

- 在 `Stop` 中关闭数据库，确保文件句柄、后台线程和缓存得到释放。

2. 实现只读快照 Reader

`Reader` 创建 `Badger` 只读事务，并用 `standAloneReader` 封装。`Reader` 的 `Close` 会丢弃事务。

理由：

- `Badger` 事务提供一致性快照。同一个 `Reader` 在其生命周期内看到稳定的数据视图。
- 这使“先创建迭代器、再发生删除”的场景仍能读取创建快照时的数据。
- 显式关闭事务可避免资源泄漏。

3. 实现列族读取与迭代

`GetCF` 通过 `engine_util.GetCFFromTxn` 读取指定列族；当 `Badger` 返回 `ErrKeyNotFound` 时转换成 `(nil, nil)`。
`IterCF` 通过 `engine_util.NewCFIterator` 创建列族迭代器。

理由：

- `TinyKV` 的 `default`、`write`、`lock` 等列族共享物理数据库，必须通过带列族前缀的物理键实现逻辑隔离。
- “键不存在”是正常业务状态，不应被当作存储错误。
- 迭代器是实现有序 `RawScan` 的基础。

4. 实现原子批量写入

`Write` 在一个 `Badger` 更新事务中遍历 `[]storage.Modify`，把 `storage.Put` 和 `storage.Delete` 分别转换成底层 `Set/Delete` 操作，并使用 `engine_util.KeyWithCF` 生成物理键。

理由：

- 一个 `WriteBatch` 内的修改需要共同提交，避免只写入一部分。
- 所有键都必须包含列族前缀，否则不同列族的同名用户键会相互覆盖。
- 对不支持的 `Modify` 类型返回错误，有助于尽早发现调用错误。

2.3 raw_api.go 的改动与理由

1. RawGet

创建 `Reader`，读取请求指定列族和键；值为 `nil` 时设置 `NotFound`，并始终关闭 `Reader`。

理由：遵守 `RawGet` 的响应语义，同时保证一次读取基于独立快照并正确释放资源。

2. RawPut 与 RawDelete

分别构造单个 `storage.Put` 或 `storage.Delete`，交给统一的 `storage.Write` 执行。

理由：RPC 层只负责协议转换，持久化、列族编码和原子提交仍由存储层统一完成。

3. RawScan

在指定列族创建迭代器，从 `StartKey` 执行 `Seek`，按字典序读取不超过 `Limit` 个键值对，并复制迭代器返回的 `key/value`。

理由：

- `Seek` 能直接定位扫描起点，避免从列族开头逐项过滤。
- `Badger` 迭代器缓冲区会被后续移动复用，因此响应中的数据必须复制。
- 限制结果数可避免一次 RPC 无界读取。

4. 错误映射

辅助函数把 `raft_storage.RegionError` 写入响应的 `RegionError`，其他错误写入普通 `Error` 字段。

理由：TinyKV RPC 协议区分路由/Region 错误和一般存储错误。虽然 Project 1 使用单机存储，这一设计也让 Raw API 能兼容后续 RaftStorage。

2.4 Project 1 的 10 个测试点

序号	测试	作用
1	<code>TestRawGet1</code>	验证 <code>RawGet</code> 能从指定列族读取已存在的值。
2	<code>TestRawGetNotFound1</code>	验证键不存在时返回 <code>NotFound</code> ，而不是伪造空值或返回存储错误。
3	<code>TestRawPut1</code>	验证 <code>RawPut</code> 后底层存储中确实存在该键值，覆盖 RPC 到存储的写入链路。
4	<code>TestRawGetAfterRawPut1</code>	验证 <code>Put</code> 后可立即 <code>Get</code> ，并验证不同列族中的同名用户键相互隔离。
5	<code>TestRawGetAfterRawDelete1</code>	验证 <code>Delete</code> 后通过 <code>RawGet</code> 查询会得到 <code>NotFound</code> 。
6	<code>TestRawDelete1</code>	直接检查底层存储，确认 <code>RawDelete</code> 真正删除了目标列族中的键。
7	<code>TestRawScan1</code>	验证扫描从 <code>StartKey</code> 开始、按字典序返回，并严格遵守 <code>Limit</code> 和对应值。
8	<code>TestRawScanAfterRawPut1</code>	验证新写入的键会出现在之后创建的扫描快照中，且顺序正确。
9	<code>TestRawScanAfterRawDelete1</code>	验证已删除键不会出现在之后的扫描中，其余键顺序保持正确。
10	<code>TestIterWithRawDelete1</code>	验证 <code>Reader/Iterator</code> 的快照隔离：迭代器创建后发生删除，旧迭代器仍能看到原快照中的键。

3. Project 4: Transactions

3.1 代码改动概览

文件	增加	删除	核心职责
<code>kv/transaction/mvcc/transaction.go</code>	105	11	MVCC 三列族读写、版本选择、当前写入查找
<code>kv/transaction/mvcc/lock.go</code>	3	3	安全收集指定事务的全部锁
<code>kv/transaction/mvcc/scanner.go</code>	70	5	合并 <code>write/lock</code> 流的事务扫描器
<code>kv/server/server.go</code>	448	13	<code>Get</code> 、 <code>Prewrite</code> 、 <code>Commit</code> 、 <code>Scan</code> 、 <code>CheckTxnStatus</code> 、 <code>Rollback</code> 、 <code>ResolveLock</code>

3.2 MVCC 数据布局

Project 4 使用三个列族表达一个事务键：

列族	键	值	用途
<code>CfDefault</code>	<code>EncodeKey(userKey, startTS)</code>	用户值	保存事务在 <code>Prewrite</code> 阶段写入的数据
<code>CfWrite</code>	<code>EncodeKey(userKey, commitTS)</code>	<code>Write{StartTS, Kind}</code>	记录已提交或已回滚的事务版本

列族	键	值	用途
CfLock	原始 use rKey	Lock {Pri mary, Ts, Ttl, Kind}	表示尚未完成第二阶段的事务锁

编码后的时间戳按倒序排列，因此对某个用户键从 `EncodeKey(key, readTS)` 开始迭代，可以找到不晚于快照时间的最新可见版本。

3.3 transaction.go 的改动与理由

1. 事务修改缓冲

`PutWrite`、`PutLock`、`PutValue` 以及对应 `Delete` 方法不直接写数据库，而是把修改加入 `txn.writes`，最后统一提交。

理由：一个事务命令经常同时修改 `lock`、`default`、`write` 列族。集中到一个 `WriteBatch` 才能保证这些变化原子生效。

2. 锁读取

`GetLock` 从 `CfLock` 读取原始用户键并反序列化，缺失时返回 `nil`。

理由：`Get`、`Prewrite`、`Commit`、`Status` 和 `Rollback` 都需要依据锁的所有者、TTL 和写入类型作判断。

3. 快照值读取

`GetValue` 从 `CfWrite` 中查找当前快照可见的最新记录：

- 只处理同一个用户键的版本；
- 跳过 `Rollback` 记录；
- 遇到 `Delete` 返回不存在；
- 遇到 `Put`，则使用该 `Write` 的 `StartTS` 到 `CfDefault` 取真实值。

理由：提交时间决定版本是否对读事务可见，而开始时间连接 `write` 记录与 `default` 中的实际值。

4. CurrentWrite 与 MostRecentWrite

`CurrentWrite` 查找与当前事务 `StartTS` 相同的 `Write`，并返回其提交时间；`MostRecentWrite` 返回某用户键最新的 `Write`。

理由：

- `CurrentWrite` 用于识别重复提交、重复回滚和“已经提交后又回滚”等状态。
- `MostRecentWrite` 用于 `Prewrite` 写冲突检查：若已有提交时间不早于本事务开始时间的版本，本事务不能继续写。

3.4 lock.go 的改动与理由

`AllLocksForTxn` 显式从空键开始 `Seek`，并使用 `KeyCopy`、`ValueCopy` 收集锁。

理由：

- 新迭代器必须先定位，否则不能可靠遍历。
- 迭代器返回的切片只在当前 `item` 生命周期内有效；复制后，`ResolveLock` 才能在迭代器前进或 `Reader` 关闭后安全使用这些键。

3.5 scanner.go 的改动与理由

Scanner 同时持有 write 迭代器和 lock 迭代器。Next 合并两个按用户键排序的数据流：

1. 从两侧选择下一个最小用户键；
2. 检查该键是否存在对当前快照可见的锁；
3. 若被锁，返回带 `KeyError` 的 pair；
4. 否则调用 `GetValue` 读取最新可见值；
5. 跳过删除或在快照中不存在的键；
6. 一次跨过同一用户键的全部 write 版本，保证每个逻辑键最多返回一次。

理由：

- 只遍历 write 列族会遗漏“只有锁、尚无 write”的键。
- 直接逐版本输出会让同一用户键重复出现。
- 合并两个有序流可在保持字典序的同时报告锁冲突并处理删除版本。

3.6 server.go 的改动与理由

1. KvGet

创建 MVCC 事务，先检查快照可见锁，再读取快照值。锁冲突写入 `KeyError`，不存在写入 `NotFound`。

理由：事务读必须同时满足版本可见性和锁可见性，不能越过未完成事务读取可能不一致的数据。

2. KvPrewrite

对 mutation key 去重并加 latch；第一遍完成写冲突、锁冲突和操作类型校验，全部通过后第二遍写入 default 值和 lock。

理由：

- 两阶段校验避免同一请求前半部分成功、后半部分冲突造成部分 Prewrite。
- latch 防止同一进程内对相同键的并发命令交错。
- 去重避免重复 mutation 或重复 latch 造成异常。

3. KvCommit

要求 `CommitVersion > StartVersion`，并逐键检查 `CurrentWrite` 和 `Lock`：

- 已回滚则返回事务终止；
- 已提交则作为幂等成功；
- 缺锁时安全跳过；
- 锁属于其他事务时返回 `retryable`；
- 自己的锁转换为 Write，并删除 Lock。

理由：Commit 必须幂等，并且不能提交已经回滚的事务或误删其他事务的锁。

4. KvScan

使用 MVCC Scanner，从 `StartKey` 开始读取到 `Limit`；普通值和逐键锁错误都放入 `Pairs`。

理由：扫描中的某个键被锁不应让整个扫描 RPC 丢失此前结果；协议允许对每个 pair 报告 `KeyError`。

5. KvCheckTxnStatus

检查 primary key 的当前 Write 和 Lock:

- 已提交: 返回提交版本;
- 已回滚: 返回零提交版本;
- 锁不存在且无 Write: 写入 Rollback, 返回 `LockNotExistRollback`;
- 锁已过期: 删除 lock/default、写 Rollback, 返回 `TTLExpireRollback`;
- 锁仍有效: 返回剩余/当前 Lock TTL, 不修改数据。

理由: 这是事务协调者恢复和清理 primary lock 的核心接口, 必须区分已完成、已过期和仍存活状态。

6. KvBatchRollback

对键去重加 latch。已提交事务拒绝回滚; 已回滚事务幂等成功; 属于本事务的 lock/default 被删除, 并写入 Rollback 标记。

理由: Rollback 标记能阻止迟到的 Prewrite/Commit 复活已取消事务; 同时不能破坏其他事务拥有的锁。

7. KvResolveLock

先扫描指定 StartVersion 的所有锁, 再对收集到的键加 latch 并重新读取; `CommitVersion > 0` 时批量提交, 否则批量回滚。

理由:

- ResolveLock 只处理仍有锁的键, 不应重写已经最终确定的记录。
- 获得 latch 后重新读取可以避免“扫描锁”和“执行处理”之间的状态变化造成 TOCTOU 竞态。

8. 统一 Reader、写入和 latch 辅助函数

新增辅助逻辑负责 Reader 创建、RegionError 映射、事务 WriteBatch 提交、空键 latch 防护和键去重。

理由: 减少各 RPC 的错误处理差异, 避免空请求死锁, 并保持 Region 错误与普通错误的协议语义。

4. Project 4A: MVCC 基础测试（15 个）

序号	测试	作用
1	TestPutLock4A	验证锁被序列化后，以原始用户键加入 CfLock 写入队列。
2	TestPutWrite4A	验证 Write 记录以 EncodeKey(key, commitTS) 加入 CfWrite。
3	TestPutValue4A	验证用户值以 EncodeKey(key, startTS) 加入 CfDefault。
4	TestGetLock4A	验证已有锁能正确解析，缺失锁返回 nil。
5	TestDeleteLock4A	验证删除锁会在 CfLock 中生成正确的 Delete 修改。
6	TestDeleteValue4A	验证按事务 StartTS 删除 CfDefault 中对应版本。
7	TestGetValueSimple4A	验证可见的已提交 Put 能通过 Write.StartTS 找到真实值。
8	TestGetValueMissing4A	验证查找缺失键时不会误读相邻用户键的版本。
9	TestGetValueTooEarly4A	验证读取时间早于 commitTS 时，该版本不可见。
10	TestGetValueOverwritten4A	验证快照位于新版本提交之后时返回最新值。
11	TestGetValueNotOverwritten4A	验证快照位于新版本提交之前时仍返回旧值。
12	TestGetValueDeleted4A	验证最新可见记录为 Delete 时返回不存在。
13	TestGetValueNotDeleted4A	验证快照早于删除提交时仍可读取旧值。
14	TestCurrentWrite4A	验证按 StartTS 找到本事务 Write 及其 commitTS，缺失时返回 nil。
15	TestMostRecentWrite4A	验证不受当前事务时间限制地取得最新 Write，并覆盖空库、缺失键和 Delete。

补充测试 TestEncodeKey 和 TestDecodeKey 不属于 -run 4A 的 15 个正式测试点，但它们分别验证 memcomparable 编码顺序、时间戳倒序，以及包含零字节的用户键能正确往返解码。

5. Project 4B: Get、Prewrite、Commit 测试（24 个）

5.1 Get（7 个）

序号	测试	作用
1	TestGetValue4B	验证 KvGet 能读取普通已提交值。
2	TestGetValueTs4B	验证相同快照时间的重复读取保持一致。
3	TestGetEmpty4B	验证空数据库读取返回 NotFound。
4	TestGetNone4B	验证目标键位于其他已有键之间但自身不存在时，不会串读相邻键。
5	TestGetVersions4B	验证多提交版本下不同快照时间的版本选择及时间边界。
6	TestGetDeleted4B	验证删除前可见、删除后不可见、后续重新写入后再可见。
7	TestGetLocked4B	验证锁之前的旧快照可读旧值，而锁之后的快照返回锁信息。

5.2 Prewrite (8 个)

序号	测试	作用
8	TestEmptyPrewrite4B	验证零 mutation 的 Prewrite 是无副作用的成功操作。
9	TestSinglePrewrite4B	验证单键 Prewrite 正确写入 value 和包含 primary、TS、TTL、kind 的 lock。
10	TestPrewriteLocked4B	验证同一键已被其他事务锁定时返回 lock conflict，并保留原锁。
11	TestPrewriteWritten4B	验证存在 commitTS >= startTS 的提交记录时产生 write conflict，且不写入部分状态。
12	TestPrewriteWrittenNoConflict4B	验证只有早于本事务开始时间的已提交版本时允许 Prewrite。
13	TestMultiplePrewrites4B	验证不同请求、不同键的 Prewrite 可以同时保留。
14	TestPrewriteOverwrite4B	验证同一请求中重复 key 的 mutation 被安全处理，最终状态符合最后一次 mutation。
15	TestPrewriteMultiple4B	验证单请求混合多个 Put/Delete 时，各键得到正确的 default/lock 状态。

5.3 Commit (9 个)

序号	测试	作用
16	TestEmptyCommit4B	验证空 key 列表提交是安全的无操作。
17	TestSingleCommit4B	验证提交删除自己的 lock、写入 Write，并保留 default 值。
18	TestCommitOverwrite4B	验证提交新版本不会破坏已有旧版本。
19	TestCommitMultipleKeys4B	验证多键提交只处理目标事务键，不影响无关已提交或仅 Prewrite 的数据。
20	TestRecommitKey4B	验证请求中重复 key 被去重，不产生重复最终记录。
21	TestCommitConflictRollback4B	验证已有 Rollback 标记的事务不能再次提交。
22	TestCommitConflictRace4B	验证锁属于其他事务时返回 retryable，且不破坏其 lock/value。
23	TestCommitConflictRepeat4B	验证相同事务重复 Commit 是幂等成功。
24	TestCommitMissingPrewrite4B	验证缺少对应 Prewrite/Lock 时 Commit 安全无操作，不影响无关数据。

6. Project 4C: Rollback、Status、Resolve、Scan 测试 (22 个)

6.1 BatchRollback (7 个)

序号	测试	作用
1	TestEmptyRollback4C	验证空 key 列表回滚是安全无操作。
2	TestRollback4C	验证删除本事务 lock/value，并写入 Rollback 标记。
3	TestRollbackDuplicateKeys4C	验证重复 key 去重，每个键只形成一个有效回滚结果。
4	TestRollbackMissingPrewrite4C	验证即使没有 Prewrite，也写 Rollback 标记以阻止迟到事务。
5	TestRollbackCommitted4C	验证已提交事务不能回滚，并保持已提交数据不变。
6	TestRollbackDuplicate4C	验证重复回滚幂等。
7	TestRollbackOtherTxn4C	验证不会删除其他事务的 lock/value，同时为当前事务写 Rollback 标记。

6.2 CheckTxnStatus (5 个)

序号	测试	作用
8	TestCheckTxnStatusTtlExpired4C	验证过期 primary lock 被清理并回滚, Action 为 TTLExpireRollback。
9	TestCheckTxnStatusTtlNotExpired4C	验证未过期 lock 保持不变, 返回有效 TTL 且不执行回滚。
10	TestCheckTxnStatusRolledBack4C	验证已有 Rollback 记录被正确识别, 不重复修改状态。
11	TestCheckTxnStatusCommitted4C	验证已有 Commit 记录被识别并返回 commitVersion。
12	TestCheckTxnStatusNoLockNoWrite4C	验证无 lock、无 write 时补写 Rollback, Action 为 LockNotExistRollback。

6.3 ResolveLock (5 个)

序号	测试	作用
13	TestEmptyResolve4C	验证不存在目标事务锁时 ResolveLock 无副作用。
14	TestResolveCommit4C	验证提交指定 StartVersion 的全部锁, 同时不影响其他事务。
15	TestResolveRollback4C	验证回滚指定 StartVersion 的全部锁, 同时不影响其他事务。
16	TestResolveCommitWritten4C	验证已提交或已回滚的最终记录不会被错误重写, 只处理实际仍存在的锁。
17	TestResolveRollbackWritten4C	验证回滚解析同样忽略已最终确定的数据, 只回滚实际锁。

6.4 Scan (5 个)

序号	测试	作用
18	TestScanEmpty4C	验证 StartKey 位于数据库末尾之后时返回空结果。
19	TestScanLimitZero4C	验证 Limit 为 0 时不返回任何 pair。
20	TestScanAll4C	验证完整扫描的字典序、快照可见性以及首尾边界。
21	TestScanLimit4C	验证 StartKey 与 Limit 的组合边界和返回顺序。
22	TestScanDeleted4C	验证删除前可见、删除后跳过、重新写入后在新快照中再次可见。

7. 测试力度评价

7.1 已覆盖得较强的部分

- Project 1 覆盖 Raw API 的增、删、查、扫、列族隔离、顺序、Limit 和 Reader 快照隔离。
- Project 4A 直接检查三列族物理布局和版本选择, 是对 MVCC 基础函数的细粒度单元测试。
- Project 4B 覆盖 Get、Prewrite、Commit 的正常路径、冲突路径、幂等、重复键、多键和版本边界。
- Project 4C 覆盖恢复类命令, 包括 TTL、缺锁回滚、已提交/已回滚状态、批量锁处理和带删除历史的扫描。
- 测试不仅判断 RPC 是否返回成功, 还会检查底层 CF 的实际状态, 因此比只检查返回码更有力度。

7.2 仍不能由这些测试证明的部分

- 没有充分覆盖进程崩溃、断电、数据库重启后的持久化与恢复。

- 没有系统性故障注入，例如批量写入中途 I/O 失败、Reader 创建失败或数据损坏。
- 并发覆盖有限，无法证明所有 latch 时序下都没有竞态、死锁或饥饿。
- 课程测试大多使用内存测试存储，不能完全替代真实 RaftStorage、Region 切分、领导者变化和网络错误。
- 对非法 CF、非法请求、损坏的 Lock/Write 编码、极大 key/value、资源压力等边界覆盖有限。
- 通过固定测试只证明“已检查场景符合预期”，不能形式化证明不存在隐藏问题。

因此，全部通过可认为实现已经满足 Project 1 和 Project 4 的主要课程功能与显式语义，可信度较高；但若要用于生产环境，还需要竞态检测、随机/模糊测试、崩溃恢复、故障注入和分布式集成测试。

8. 最终结论

Project 1 的核心价值是建立了正确的单机存储抽象：Badger 持久化、列族隔离、快照 Reader、原子 WriteBatch 和完整 Raw API。

Project 4 在该存储抽象上实现了 Percolator 风格事务：default/write/lock 三列族、快照读、Prewrite/Commit 两阶段提交、幂等回滚、事务状态恢复、批量锁解析以及 MVCC Scan。

从代码变化和 71 个正式测试点看，实现与课程设计目标是一致的。现有测试足以作为项目验收的重要依据，但不应被解释为对所有并发、崩溃和分布式场景的绝对正确性证明。